A Source-Available ESP32 Testbed for PSK-Authenticated X25519 and ML-KEM-768 Key Exchange

Hafiz Ali Mansoor Elahi[1*] and Muhammad Sohaim Muqtada[2]

[1*]Promaker Institute, 18-C Block, Civic Center, Faisal Town, Lahore, Pakistan.

[2]Department of Cybersecurity, University of Management and Technology, C-II, Johar Town, Lahore, Pakistan.

*Corresponding author(s). E-mail(s): hafiz.ali@promakerinstitute.com; ORCID: 0009-0006-4378-6391;
Contributing authors: f2024408084@umt.edu.pk;

Abstract

Purpose: To characterize a pre-provisioned, PSK-authenticated application-layer exchange combining X25519 and ML-KEM-768 on an ESP32. **Methods:** We document a FreeRTOS client, Python HTTP server, native C KEM and encrypted telemetry lifecycle. We reanalyse 100 archived ESP32-D0WD-V3 records at 160 MHz, keeping primitive, TLS and heap measurements as separate acquisitions. **Results:** The 100 records marked HMAC-verified completion have mean instrumented latency 616.75 ms (SD 244.13 ms; 95% CI 568.31 to 665.19 ms) and mean elapsed CCOUNT of 100.33 million cycles; the attempted-run denominator is unavailable. A separate primitive trace gives 592.80 ms for the summed X25519 operation means and 32.47 ms for the client ML-KEM-768 operations. These are not matched before-and-after or end-to-end comparisons. A separate, incomplete heap profile reports 13.8 KB and is not a campaign result. **Conclusion:** The contribution is a source-available testbed and bounded implementation characterization. It is not a new primitive or combiner, a first-deployment claim, or evidence of advantage over TLS. Shared bidirectional traffic keys, incomplete historical binary provenance and unevaluated side channels constrain security conclusions.

Keywords: ESP32, ML-KEM, X25519, FreeRTOS, Reproducibility, Telemetry

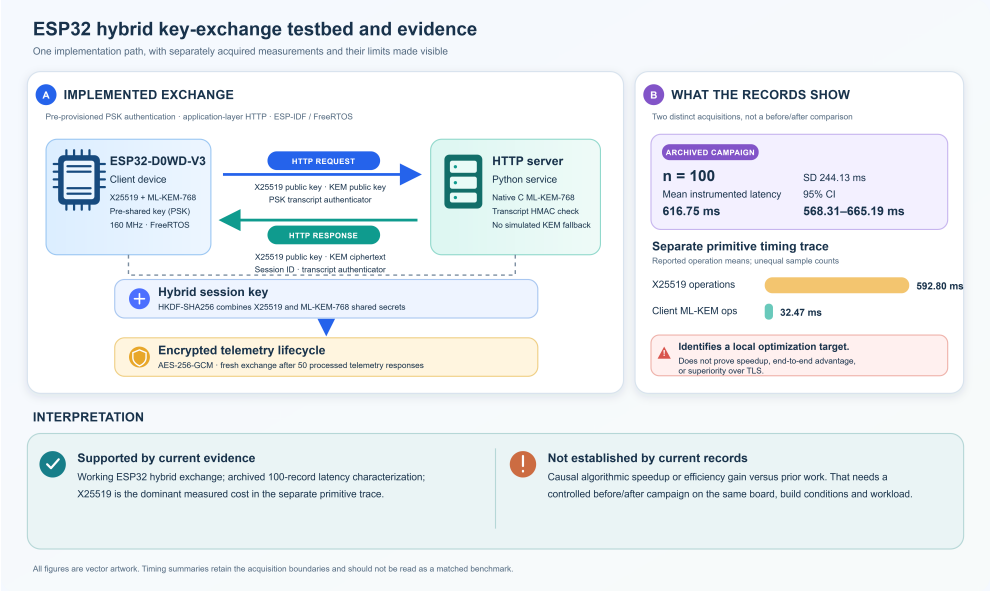


Fig. 1 Protocol and evidence overview. Panel A shows the application exchange and telemetry lifecycle implemented in the testbed. Panel B separates the $n = 100$ campaign from the primitive-operation trace and the independent, incomplete heap profile. The primitive bars show means from their own acquisition and do not imply a matched end-to-end comparison or a measured speedup over TLS. Here, pk denotes public key, ct denotes ciphertext, and HMAC denotes transcript authentication.

1 Introduction and research scope

Embedded post-quantum experiments need more than primitive timing: readers must be able to inspect the network messages, authentication assumptions, task configuration and measurement boundaries. The released software [1] exposes these elements for a pre-provisioned, PSK-authenticated application-layer exchange on an ESP32-D0WD-V3 at 160 MHz. It combines X25519 [2, 3] and ML-KEM-768 [37] in a FreeRTOS client and asynchronous Python HTTP service. Figure 1 separates the implemented protocol path from the independently acquired evidence. The paper's contribution is this inspectable deployment and its bounded characterization, not a new KEM or a claim that PQC first became feasible on ESP32.

X-Wing already combines X25519 and ML-KEM-768 using a specified combiner and security analysis [38]. Earlier work evaluated post-quantum TLS on embedded systems [39]; recent studies examine post-quantum IoT authentication over TLS, hybrid ML-KEM-768/X25519 MQTT, hybrid TLS/OPC UA deployments, and TLS 1.3 PQC benchmarks that include ESP32 [27, 43–45]. These are close precedents, so the present paper does not claim algorithmic novelty, first use of hybrid PQC on ESP32, or general superiority. Its distinct scope is a certificate-free, pre-provisioned-PSK HTTP exchange with an encrypted telemetry lifecycle, documented as an inspectable implementation and evaluated through clearly separated acquisitions. Table 1 compares the

settings and limits of inference; it is a purposive scope comparison, not a systematic review or a harmonized quantitative benchmark. Neither X-Wing's proof nor TLS security properties transfer automatically to this construction.

Table 1 Comparison with selected related implementations and evaluations. The studies use different protocols, authentication, libraries and measurement boundaries; numerical results are therefore not pooled or ranked.

Study	System and method	Relevance and comparison boundary
Krusniauskas et al. [43]	TLS authentication overhead across classical, full-PQC and hybrid settings.	TLS cost boundaries differ from this PSK-authenticated HTTP measurement.
Saleem et al. [44]	PQ-MQTT hybrid X25519/ML-KEM-768 protocol for IoT networks.	Closest design family; the hybrid mix itself is not new. MQTT and measurement conditions differ, so no direct latency ranking is made.
Iqbal et al. [45]	Hybrid-PQC TLS telemetry and OPC UA SecureChannel scenarios.	TLS/OPC UA, parameters and authentication path differ.
Gillot et al. [27]	TLS 1.3 PQC benchmark matrix across constrained devices, including ESP32.	Similar device family; TLS stack and measurements are unmatched.
Huang et al. [23]	Plantard-arithmetic optimization of Kyber on low-end 32-bit IoT processors.	An algorithm-level optimization example, but with a different KEM/CPU. No equivalent matched before/after change is demonstrated here.
This study	PSK-authenticated HTTP exchange and AES-GCM telemetry on ESP32-D0WD-V3; archived $n = 100$ campaign plus separate primitive and heap acquisitions.	Source-visible characterization only; no matched competitor, before/after experiment or general speedup is claimed.

The package is useful for examining binary transcript framing, measurement placement and the distinction between historical hardware observations and current host checks. The extended technical analysis and historical comparisons are provided in Online Resource 1. Source code, raw evidence, analysis scripts and reproduction instructions are provided in Online Resource 2. Separate data acquisitions remain explicitly identified.

2 Related work

2.1 Post-quantum transition and standardized KEMs

The prospect of quantum algorithms affecting public-key cryptography has motivated migration planning as well as new cryptographic constructions [4–7]. NIST's standardization work led to ML-KEM, standardized in FIPS 203; the associated recommendations describe KEM use and key-establishment practice [12, 14, 37]. ML-KEM is based on the module-lattice line of work developed from learning-with-errors constructions [8–11]. The Fujisaki-Okamoto transform is relevant background to CCA-secure KEM design, but the security claims in this paper remain limited to the standardized ML-KEM component and the properties actually modeled [28]. These sources establish the cryptographic and migration context, but they do not imply that

an application using a standardized KEM inherits a protocol-level security proof. This study therefore evaluates the documented integration and its evidence boundaries.

2.2 Hybrid key establishment and protocol standards

Hybrid designs combine classical and post-quantum key-establishment material so that security can be retained under stated assumptions about at least one component; their security depends on the combiner, transcript binding and protocol context [13, 15, 16, 38]. The IETF has since published a general TLS 1.3 hybrid key-exchange construction and concrete X25519MLKEM768 groups [17, 18], alongside the TLS 1.3 protocol specification [19]. Those standardized mechanisms have a different wire format, negotiation process and authentication context from the application-layer exchange studied here. In particular, this paper concatenates component outputs and applies its documented HKDF profile; it does not claim conformance to X-Wing or the TLS hybrid groups. The KEM and key-derivation interfaces are discussed in their own standards and guidance [14, 29, 41].

2.3 Embedded implementations and post-quantum TLS

Prior implementation work has optimized lattice-based cryptography for constrained processors, including Cortex-M4 implementations and arithmetic techniques for low-end 32-bit IoT devices [20–23]. Other work has integrated post-quantum algorithms into embedded TLS stacks and examined TLS authentication or handshake costs [24–26, 39]. These studies provide important implementation and protocol baselines; their platforms, algorithm versions, authentication paths, libraries and measurement methods differ, so their numerical results are not pooled with the present campaign.

Recent IoT studies have evaluated post-quantum authentication and hybrid communications in TLS, MQTT and OPC UA settings [43–45]. Especially close in platform and algorithm choice, Gillot et al. report a TLS 1.3 benchmark matrix that includes ESP32 and X25519/ML-KEM combinations [27]. Their preprint makes clear that implementation and architecture can materially affect relative costs. It also means this manuscript cannot claim that it is the first ESP32 evaluation of X25519 plus ML-KEM or that a custom exchange is generally more efficient than PQ-TLS. The present work has a narrower purpose: to expose a pre-provisioned-PSK HTTP exchange, its telemetry lifecycle and its source-level measurement boundaries, then report archived observations without treating them as a matched comparison to TLS or the recent benchmark matrix.

2.4 Implementation security and evaluation boundaries

Authenticated encryption, message authentication and key derivation each have established interfaces and guidance [30–32, 34, 40, 46]. Formal protocol analysis can test specified properties of an abstract model, while implementation-level leakage requires separate analysis; published lattice-cryptography side-channel studies illustrate why source-level intent alone cannot establish resistance [35, 36]. We use ProVerif for the limited properties described in Section 5, and do not infer protection against later PSK compromise, concrete nonce misuse, device memory exposure or physical side

channels. This distinction places the work alongside prior cryptographic and embedded evaluations without claiming those properties were experimentally demonstrated here.

2.5 Research questions and study design

The study asks three questions: which client operations dominate the cost of the selected hybrid implementation; what latency distribution and elapsed-cycle behavior occur in the archived application campaign; and how the authentication, framing and measurement choices constrain interpretation of these results? We examine one ESP32 platform and one archived campaign. The analysis retains every recorded observation, identifies the measurement intervals in the source, and distinguishes observations from symbolic and host-level checks. It is a descriptive implementation study; it does not test a causal performance advantage over TLS or establish a new cryptographic construction.

2.6 Engineering objective and contributions

The intended deployment consists of pre-provisioned nodes whose peers already share authentication credentials. The engineering objective is to characterize a certificate-free hybrid exchange integrated with Wi-Fi, an RTOS task and encrypted telemetry. Omitting certificate processing changes the authentication and provisioning model; a lower observed cost cannot automatically be attributed to a better cryptographic implementation.

The study makes three bounded contributions. First, it provides an inspectable integration of hybrid exchange, PSK transcript authentication and periodic telemetry-session renewal for a pre-provisioned ESP32 deployment. Second, it reports an operation-level trace that identifies the selected X25519 implementation as the principal local optimization target. Third, it reanalyses an archived application campaign while distinguishing it from separate primitive, TLS and heap acquisitions. In the primitive trace, the two measured X25519 operation means sum to 592.80 ms, about 18.3 times the 32.47 ms sum of the two measured client-side ML-KEM-768 operations. This trace-specific observation is not a matched end-to-end comparison and does not show that X25519 is inherently slower.

3 Implementation and protocol

Table 2 identifies the principal components. The firmware uses ESP-IDF and mbedTLS interfaces, with a repository-native C implementation for ML-KEM-768. The server uses Python's `cryptography` package and loads the native shared library. An unavailable native backend causes PQC operations to fail; no simulated key material is substituted.

The configuration defines classical, PQC-only and hybrid modes, but the principal archived campaign examined here is hybrid. The current application invokes 100 hybrid attempts, then continuous telemetry. The configured task stack is 32,768 bytes. Stack capacity, stack high-water readings, static memory and dynamic heap allocations are

Table 2 Components and reproducibility interfaces.

Component	Entry point or responsibility
Client	`firmware/main/main.c`: Wi-Fi, campaign and telemetry task
Cryptographic adapter	`crypto_hybrid.c`: key generation, serialization, response validation and telemetry
Native KEM	`firmware/components/mlkem768`: C sources and host Makefile
Server	`server/server.py`: `/handshake` and `/api/telemetry`
Analysis	`tools/analyze_submission.py`: raw/CSV consistency and statistics
Host checks	`tests/test_submission_fixes.py`: transcript validation and missing-backend behavior

distinct quantities. The archived campaign records ESP-IDF v5.5; its complete build configuration and exact firmware binary are not available.

3.1 Handshake encoding and key derivation

The client sends one binary request to `/handshake`. Table 3 gives the hybrid body fields. HTTP headers and TCP/IP overhead are excluded from the listed lengths. The application exchange has one request and one response after transport establishment. The campaign creates and cleans up an HTTP client per handshake; persistent connection reuse is not established.

Table 3 Hybrid application-body encoding, in transmission order.

Body	Fields and byte lengths	Total
Request	Mode (1), client X25519 public key (32), KEM public key (1,184), HMAC (32)	1,249
Response	Session ID (16), server X25519 public key (32), KEM ciphertext (1,088), HMAC (32)	1,168
Telemetry	Session ID (16), IV (12), ciphertext (variable), GCM tag (16)	Variable

HMAC-SHA256 [40] authenticates the request's first 1,217 bytes. The response HMAC covers those same unsigned request bytes followed by the response's Session ID, server public key and KEM ciphertext. It excludes both authentication tags. Verification therefore binds the server response to the current client parameters. The server authenticates possession of the PSK in the request; confirmation that the client derived the same traffic key arrives through valid telemetry, without a separate client-finished message.

The shared input is the 32-byte X25519 secret followed by the 32-byte KEM secret. HKDF-SHA256 [29, 41] produces 32 bytes using salt `HybridPQC-ESP32-Session-v1` and info `session-key`. The PSK authenticates the transcript and is not input entropy

to HKDF. The published evaluation PSK is public and offers no private-credential authentication to someone who can read the repository. A security experiment must provision private credentials separately.

Before concatenating the secrets, the firmware scans the X25519 output and rejects an all-zero value as required by RFC 7748 [2]. The host test suite checks the accumulator logic and server-side rejection of an all-zero input point. These checks are not a hardware integration test of the corrected firmware; the archived campaign predates the later harness corrections.

The server creates the 16-byte Session ID using Python's `secrets.token_bytes(16)` and the operating-system CSPRNG. The client uses ESP-IDF randomness for its local operations; true-random output depends on the documented entropy-source conditions [42]. A Session ID is an identifier, not a substitute for transcript authentication.

3.2 Telemetry and task lifecycle

Telemetry uses AES-256-GCM [31–33, 46] without additional authenticated data. Its 12-byte IV contains a four-byte counter followed by eight random bytes. The server checks that received request counters advance. After 50 successfully processed telemetry responses, the client leaves the loop, cleans up the context and starts a fresh handshake. The archived rotation trace supports historical behavior, not execution of the later corrected firmware.

Both directions currently use the same traffic key. Counter checks do not bind a message to a sending role, so directional reflection and cross-direction replay remain limitations. The software is intended for protocol inspection and laboratory evaluation. It should not be used as a secure Internet deployment without redesign and validation of these boundaries.

4 Reproduction methods and implementation checks

The code is distributed under the repository's MIT license; dependency licenses remain applicable. The native host build uses a Linux/macOS-compatible C toolchain and Make. Python dependencies are listed in `server/requirements.txt`. ESP32 reproduction requires ESP-IDF v5.5, a suitable board and a reachable Wi-Fi/server configuration. Commands and metadata requirements are supplied in `docs/reproduction_steps.md` and S2.

1. Check out the exact frozen source version. Recompute historical statistics using the check mode of `tools/analyze_submission.py`, as documented in the reproduction guide. This compares numerical records with the UART trace and verifies the generated manuscript values.
2. Build the native library with `make -C firmware/components/mlkem768`. Install the declared Python requirements, then run `python3 -m unittest discover -s tests` and `python3 tools/test_mlkem768_kat.py`.

3. Start `python3 server/server.py` on its default port 8443. Configure local Wi-Fi and server settings in the ignored `wifi_config.local.h`; keep private credentials out of published files.
4. In an ESP-IDF terminal, select target `esp32`, build, flash and capture the full serial output. Preserve the exact commit, toolchain, configuration, ELF/map and binary hashes. Retain all attempts and failures, rather than selecting only completed fast runs.

The legacy `test_mlkem768_kat.py` filename is retained for compatibility. It checks fixed-seed and randomized round trips and modified ciphertext behavior. It does not compare authoritative external expected-answer vectors and does not establish NIST certification. Current Windows host checks exercise both server entry points' framing/HMAC behavior with a substituted KEM output, and missing-library failure behavior. These checks passed in the preparation environment; they are not native KEM or hardware measurements. Full native-library tests require a compatible toolchain and runtime.

The current harness preserves attempted-run identifiers and computes successful-sample statistics separately. It also removes invalid watchdog-reset calls from an unsubscribed task. Host regression checks address failure accounting, but do not validate scheduling or watchdog behavior on an ESP32. A new hardware campaign must be saved separately if these later corrections are to be called physically validated.

OpenAI Codex assisted with evidence-consistency checks, analysis scripting, source and documentation edits, manuscript editing and preparation of Fig. 1. It generated no hardware measurements. The authors are responsible for independently reviewing and validating the final manuscript, all cited claims and all reported values before submission.

5 Archived results and validation boundaries

Table 4 summarizes the reanalysis of 100 archived hybrid handshake records [47]. All logged observations, including the latency tail, are retained. The instrumented latency sums two `esp_timer_get_time()` intervals covering key generation and network exchange plus response processing. Intervening packing, logging and heap sampling are excluded. The broader CCOUNT interval includes waits and instrumentation and measures elapsed cycles, rather than isolated processor work. Subtracting these intervals would not yield a valid estimate of networking overhead.

5.1 Primitive cost and exploratory TLS observations

A separate UART acquisition, `energy_esp32_mlkem768_usb_proof.txt`, contains repeated primitive timer and cycle observations. Direct parsing gives the means in Table 5. Sample counts differ by operation; this trace is separate from the 100-handshake campaign.

The sums of the two X25519 and two client-side ML-KEM operation means are 592.80 and 32.47 ms, respectively, a ratio of approximately 18.3:1 in this primitive

Table 4 Reanalysis of the historical hybrid campaign ($n = 100$).

Quantity	Value
Mean instrumented latency	616.75 ms
Sample standard deviation	244.13 ms
95% CI of the mean	568.31 to 665.19 ms
Minimum / maximum	525.66 / 1989.39 ms
Mean elapsed CCOUNT	100.33 million
Logged HMAC-verified completions	100/100; all-attempt denominator not preserved
Campaign transient heap peak	Not established

Table 5 Client primitive measurements from a separate acquisition.

Operation	Mean (ms)	n
X25519 key generation	392.27	50
X25519 shared secret	200.53	49
ML-KEM-768 key generation	13.89	50
ML-KEM-768 decapsulation	18.58	49

trace. This identifies the selected X25519 implementation as the principal local optimization target. The sums use unequal sample counts, exclude server encapsulation and are not measured fractions of an individual complete handshake. They must not be subtracted from the separately acquired campaign mean to estimate network overhead or presented as an algorithmic comparison.

The archive also contains real, but small and separate, TLS observations. Two wolfSSL TLS 1.3 exchanges explicitly negotiate `X25519MLKEM768`, with observed durations 1092.763 and 1101.846 ms (mean 1097.30 ms, $n = 2$). Four classical TLS observations span 798.797 to 2350.741 ms. A separate custom-protocol excerpt records 849.604 ms. These runs document that the experiments were performed; they are exploratory context only because sample counts, authentication, acquisition conditions and build records are not matched. They do not establish a controlled percentage speedup or peak-memory saving over TLS. Before/after free-heap snapshots do not measure transient peak allocation.

The confidence interval uses Student's t distribution with 99 degrees of freedom and sample SD. It assumes independent observations within this campaign and does not quantify variability across devices or environments. The 100 completed records describe the archived acquisition window; they do not establish the later harness's all-attempt success rate. The archive contains 100 watchdog API "task not found" errors, disclosed separately from handshake completion. These log messages do not prove hardware watchdog resets.

A separate heap profile reports a peak dynamic allocation of 13.8 KB, excluding the configured task stack and static storage; the underlying transient trace is incomplete.

This profile is independent of the $n = 100$ campaign. Campaign snapshots average 22.24 bytes and reach 440 bytes, but miss transient allocations and are not a campaign peak. Primitive timings, manual electrical notes and TLS traces were collected separately. They do not form a controlled, authentication-equivalent efficiency comparison. Energy values derived from fixed windows are not synchronized power integrals.

Archived ProVerif output [48] establishes encoded client/server payload confidentiality and client-to-server injective agreement under idealized operations and an uncompromised private PSK. It does not query later PSK disclosure, reverse authentication, concrete GCM nonces or physical memory exposure. Selected client-buffer zeroization is not evidence of complete server-side erasure or resistance to a live memory dump. Empirical side-channel resistance remains unevaluated.

The recorded firmware version is `v2.0-mlkem768-final-6-g45d5f57-`; its ELF digest is truncated, so an exact historical binary-to-source mapping is unavailable. Synthetic PCAP illustrations remain illustrative and are not wire captures. The recovered master tcpdump text is now included in the audit materials (SHA-256: `958cf70de291297887b70c49e2e6cb8c90e44fc6f5b48036c4da7758867c4494`); an independent parser reproduces the supplied count and port totals of 14,157 endpoint-flow records (10,235 on port 8443 and 3,922 on port 4443). The raw file records 17,502 client SYN events, but SYN events and parser-accepted TCP flows are not cryptographic attempts or successful handshakes. Port numbers alone do not identify TLS mode, device, or firmware, and the capture does not independently establish uninterrupted operation or a success rate. S1 preserves these limits and the separate acquisition boundaries.

6 Discussion and conclusions

The testbed provides source-level access to the PSK-authenticated exchange, its telemetry lifecycle and its historical measurement analysis. Researchers can reuse its encodings and analysis interfaces to study implementation changes while preserving acquisition provenance. A new experiment should identify its own build, failures and raw observations, and use matched authentication and instrumentation when comparing protocol costs.

The primitive acquisition identifies the selected X25519 implementation, rather than client-side ML-KEM, as the dominant local cost in that trace (592.80 ms against 32.47 ms for the two measured ML-KEM operations). This provides a concrete optimization priority for this software configuration, although a matched before-and-after experiment is required to demonstrate an improvement. The historical campaign yields a mean instrumented latency of 616.75 ms and mean elapsed CCOUNT of 100.33 million cycles on the stated ESP32 setup. Together these records characterize one certificate-free, PSK-authenticated application-layer prototype; they do not show superiority to TLS. Direction-separated traffic keys, stronger implementation correctness checks and controlled memory/energy measurements are necessary for stronger security or efficiency claims. No production-readiness, first-implementation or new-primitive claim is made.

7 Data and code availability

The implementation and supporting evidence are identified by source commit b4b718cd359f0c134fc7bfc619172c01bdf6eb49 in the https://github.com/Hybrid-PQC-IoT-Lab/esp32-hybrid-pqc-key-exchange repository (pinned commit record: https://github.com/Hybrid-PQC-IoT-Lab/esp32-hybrid-pqc-key-exchange/commit/b4b718cd359f0c134fc7bfc619172c01bdf6eb49). The full commit SHA identifies this source snapshot. This commit identifies source and evidence, not the hardware build that produced the historical measurements. The 100-record historical campaign and its analysis source are separately preserved at commit 467bbe7a3c5a718d4eec7c882bc00f042487304d, under https://github.com/Hybrid-PQC-IoT-Lab/esp32-hybrid-pqc-key-exchange/tree/467bbe7a3c5a718d4eec7c882bc00f042487304d/docs/evidence. The recovered endurance tcpdump text and neutral packet-level audit are included in the accompanying evidence materials; the raw capture SHA-256 is `958cf70de291297887b70c49e2e6cb8c90e44fc6f5b48036c4da7758867c4494`. A separate corrected-source ESP-IDF v5.5 compiler archive is supplied with its original commit and hashes; it does not identify the historical measurement binary. No new board execution is implied. Original source copyright and license notices are retained.

Supplementary information

Online Resource 1: Extended technical analysis, including protocol details and limitations of the separate historical TLS, endurance, heap and electrical acquisitions.

Online Resource 2: Source code, raw records, analysis scripts, verification output, reproduction instructions and checksums associated with the cited source release.

Statements and Declarations

Author contributions

Hafiz Ali Mansoor Elahi: Conceptualization, Methodology, Supervision, Validation. Muhammad Sohaim Muqtada: Software, Investigation, Data curation.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interests

Hafiz Ali Mansoor Elahi is a co-founder of Promaker Institute, his stated institutional affiliation. The authors declare no competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

OpenAI Codex assisted with evidence-consistency checks, analysis scripting, code and documentation edits, manuscript editing and preparation of the schematic overview figure. It generated no hardware measurements. The authors take responsibility for the final content; further details are provided in the methods section.

Ethics approval

This study reports software and hardware experiments and did not involve human participants, personal participant data or animals. Ethics approval and participant consent are not applicable.

References

[1] H. A. M. Elahi and M. S. Muqtada, "ESP32 hybrid key-exchange testbed," source and evidence edition [software], GitHub, 2026. Source commit b4b718cd359f0c134fc7bfc619172c01bdf6eb49. Available: https://github.com/Hybrid-PQC-IoT-Lab/esp32-hybrid-pqc-key-exchange/commit/b4b718cd359f0c134fc7bfc619172c01bdf6eb49.

[2] A. Langley, M. Hamburg, and S. Turner, "Elliptic Curves for Security," RFC 7748, Jan. 2016, doi: 10.17487/RFC7748. [Online]. Available: https://www.rfc-editor.org/rfc/rfc7748

[3] D. J. Bernstein, "Curve25519: New Diffie-Hellman speed records," in *PKC 2006*, LNCS 3958, pp. 207 to 228, doi: 10.1007/11745853_14. [Online]. Available: https://doi.org/10.1007/11745853_14

[4] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proc. 35th FOCS*, 1994, pp. 124 to 134, doi: 10.1109/SFCS.1994.365700. [Online]. Available: https://doi.org/10.1109/SFCS.1994.365700

[5] L. Chen *et al.*, "Report on Post-Quantum Cryptography," NIST IR 8105, 2016, doi: 10.6028/NIST.IR.8105. [Online]. Available: https://doi.org/10.6028/NIST.IR.8105

[6] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38 to 41, 2018, doi: 10.1109/MSP.2018.3761723. [Online]. Available: https://doi.org/10.1109/MSP.2018.3761723

[7] D. Moody, R. Perlner, A. Regenscheid, A. Robinson, and D. Cooper, "Transition to Post-Quantum Cryptography Standards," NIST IR 8547, Oct. 2026, doi: 10.6028/NIST.IR.8547. [Online]. Available: https://csrc.nist.gov/pubs/ir/8547/final

[8] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, pp. 1 to 40, 2009, doi: 10.1145/1568318.1568324.

[Online]. Available: https://doi.org/10.1145/1568318.1568324

[9] J. Bos *et al.*, "CRYSTALS-Kyber: A CCA-secure module-lattice-based KEM," in *Proc. IEEE EuroS&P*, 2018, pp. 353 to 367, doi: 10.1109/EuroSP.2018.00032. [Online]. Available: https://doi.org/10.1109/EuroSP.2018.00032

[10] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *EUROCRYPT 2010*, LNCS 6110, pp. 1 to 23, doi: 10.1007/978-3-642-13190-5_1. [Online]. Available: https://doi.org/10.1007/978-3-642-13190-5_1

[11] R. Avanzi *et al.*, "CRYSTALS-Kyber algorithm specifications and supporting documentation (version 3.02)," NIST PQC Round 3, 2021. [Online]. Available: https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf

[12] G. Alagic *et al.*, "Status Report on the Third Round of the NIST Post-Quantum Cryptography Standardization Process," NIST IR 8413, 2022, doi: 10.6028/NIST.IR.8413. [Online]. Available: https://doi.org/10.6028/NIST.IR.8413

[13] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, and D. Stebila, "Hybrid key encapsulation mechanisms and authenticated key exchange," in *PQCrypto 2019*, LNCS 11505, pp. 206 to 226, doi: 10.1007/978-3-030-25510-7_12. [Online]. Available: https://doi.org/10.1007/978-3-030-25510-7_12

[14] G. Alagic *et al.*, "Recommendations for Key-Encapsulation Mechanisms," NIST SP 800-227, 2025, doi: 10.6028/NIST.SP.800-227. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-227

[15] F. Driscoll, M. Parsons, and B. Hale, "Terminology for Post-Quantum Traditional Hybrid Schemes," RFC 9794, Jun. 2025, doi: 10.17487/RFC9794. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9794

[16] R. Barnes, K. Bhargavan, B. Lipp, and C. Wood, "Hybrid Public Key Encryption," RFC 9180, Feb. 2022, doi: 10.17487/RFC9180. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9180

[17] K. Kwiatkowski, P. Kampanakis, B. E. Westerbaan, and D. Stebila, "Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3," RFC 10024, Aug. 2026, doi: 10.17487/RFC10024. [Online]. Available: https://www.rfc-editor.org/info/rfc10024/

[18] D. Stebila, S. Fluhrer, and S. Gueron, "Hybrid Key Exchange in TLS 1.3," RFC 9954, Jul. 2026, doi: 10.17487/RFC9954. [Online]. Available: https://www.rfc-editor.org/info/rfc9954/

[19] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 8446, Aug. 2018, doi: 10.17487/RFC8446. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446

[20] L. Botros, M. J. Kannwischer, and P. Schwabe, "Memory-efficient high-speed implementation of Kyber on Cortex-M4," in *AFRICACRYPT 2019*, LNCS 11627, pp. 209 to 228, doi: 10.1007/978-3-030-23696-0_11. [Online]. Available: https://

doi.org/10.1007/978-3-030-23696-0_11

[21] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4," IACR ePrint 2019/844. [Online]. Available: https://eprint.iacr.org/2019/844

[22] A. Abdulrahman, V. Hwang, M. J. Kannwischer, and A. Sprenkels, "Faster Kyber and Dilithium on the Cortex-M4," in *ACNS 2022*, LNCS 13269, pp. 853 to 871, doi: 10.1007/978-3-031-09234-3_42. [Online]. Available: https://doi.org/10.1007/978-3-031-09234-3_42

[23] J. Huang *et al.*, "Yet another improvement of Plantard arithmetic for faster Kyber on low-end 32-bit IoT devices," *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 3800 to 3813, 2024, doi: 10.1109/TIFS.2024.3371369. [Online]. Available: https://doi.org/10.1109/TIFS.2024.3371369

[24] P. Schwabe, D. Stebila, and T. Wiggers, "Post-quantum TLS without handshake signatures," in *Proc. ACM CCS*, 2020, pp. 1461 to 1480, doi: 10.1145/3372297.3423350. [Online]. Available: https://doi.org/10.1145/3372297.3423350

[25] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking post-quantum cryptography in TLS," in *PQCrypto 2020*, LNCS 12100, pp. 72 to 91, doi: 10.1007/978-3-030-44223-1_5. [Online]. Available: https://doi.org/10.1007/978-3-030-44223-1_5

[26] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-Quantum Authentication in TLS 1.3: A Performance Study," in *Proc. NDSS*, 2020, doi: 10.14722/ndss.2020.24203. [Online]. Available: https://doi.org/10.14722/ndss.2020.24203

[27] O. Gillot, W. J. Buchanan, and C. Turino, "Testing TLS 1.3 PQC on constrained devices and microcontrollers," arXiv:2610.27766, submitted Oct. 2026, accepted at QSIoT 2026 workshop, doi: 10.48550/arXiv.2610.27766. [Online]. Available: https://arxiv.org/abs/2610.27766

[28] E. Fujisaki and T. Okamoto, "Secure integration of asymmetric and symmetric encryption schemes," in *CRYPTO '99*, LNCS 1666, pp. 537 to 554, doi: 10.1007/3-540-48405-1_34. [Online]. Available: https://doi.org/10.1007/3-540-48405-1_34

[29] E. Barker, L. Chen, and R. Davis, "Recommendation for Key-Derivation Methods in Key-Establishment Schemes," NIST SP 800-56C Rev. 2, 2020, doi: 10.6028/NIST.SP.800-56Cr2. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-56Cr2

[30] NIST, "FIPS 180-4: Secure Hash Standard (SHS)," 2015, doi: 10.6028/NIST.FIPS.180-4. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.180-4

[31] D. McGrew and J. Viega, "The security and performance of the Galois/Counter Mode (GCM) of operation," in *INDOCRYPT 2004*, LNCS 3348, pp. 343 to 355, doi: 10.1007/978-3-540-30556-9_27. [Online]. Available: https://doi.org/10.1007/978-3-540-30556-9_27

[32] D. McGrew, "An Interface and Algorithms for Authenticated Encryption," RFC 5116, Jan. 2008, doi: 10.17487/RFC5116. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5116

[33] NIST, "FIPS 197: Advanced Encryption Standard (AES)," Nov. 2001, doi: 10.6028/NIST.FIPS.197. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.197

[34] NIST, "FIPS 198-1: The Keyed-Hash Message Authentication Code (HMAC)," 2008, doi: 10.6028/NIST.FIPS.198-1. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.198-1

[35] P. Ravi, S. Sinha Roy, A. Chattopadhyay, and S. Bhasin, "Generic side-channel attacks on CCA-secure lattice-based PKE and KEMs," *IACR TCHES*, vol. 2020, no. 3, pp. 307 to 335, 2020, doi: 10.13154/tches.v2020.i3.307-335. [Online]. Available: https://doi.org/10.13154/tches.v2020.i3.307-335

[36] D. J. Bernstein *et al.*, "KyberSlash: Exploiting secret-dependent division timings in Kyber implementations," *IACR TCHES*, vol. 2025, no. 2, pp. 209 to 234, doi: 10.46586/tches.v2025.i2.209-234. [Online]. Available: https://doi.org/10.46586/tches.v2025.i2.209-234

[37] NIST, "FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard," Aug. 2024, doi: 10.6028/NIST.FIPS.203. [Online]. Available: https://doi.org/10.6028/NIST.FIPS.203

[38] M. Barbosa, D. Connolly, J. D. Duarte, A. Kaiser, P. Schwabe, K. Varner, and B. Westerbaan, "X-Wing: The Hybrid KEM You've Been Looking For," *IACR Communications in Cryptology*, vol. 1, no. 1, Apr. 2024, doi: 10.62056/a3qj89n4e. [Online]. Available: https://eprint.iacr.org/2024/039

[39] K. Bürstinghaus-Steinbach, C. Krauß, R. Niederhagen, and M. Schneider, "Post-quantum TLS on embedded systems: Integrating and evaluating Kyber and SPHINCS+ with mbed TLS," in *Proc. ACM ASIACCS*, 2020, pp. 841 to 852, doi: 10.1145/3320269.3384725. [Online]. Available: https://doi.org/10.1145/3320269.3384725

[40] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-Hashing for Message Authentication," RFC 2104, Feb. 1997, doi: 10.17487/RFC2104. [Online]. Available: https://www.rfc-editor.org/rfc/rfc2104

[41] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," RFC 5869, May 2010, doi: 10.17487/RFC5869. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5869

[42] Espressif Systems, "Random Number Generation," ESP-IDF Programming Guide v5.5, accessed Oct. 5, 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32/api-reference/system/random.html

[43] S. Krusniauskas, S. Grigaliunas, and R. Bruzgiene, "Authentication Overhead in Post-Quantum IoT for Classical, Full-PQC and Hybrid TLS," in *Proc. 2026 IEEE 13th Workshop on Advances in Information, Electronic and Electrical Engineering*, 2026, doi: 10.1109/AIEEE71113.2026.11577039. [Online]. Available:

https://doi.org/10.1109/AIEEE71113.2026.11577039

[44] S. Saleem, S. M. Umran, Z. A. Abduljabbar, P. Perazzo, and U. Saleem, "PQ-MQTT: A Hybrid Post-Quantum Secure Communication Protocol for MQTT-Based IoT Networks," in *Proc. 2026 IEEE 2nd International Conference on Secure IoT, Assured and Trusted Computing (SATC)*, 2026, doi: 10.1109/SATC69565.2026.11542264. [Online]. Available: https://doi.org/10.1109/SATC69565.2026.11542264

[45] M. Iqbal, A. L. B. Pereda, A. Pellegata, D. Gemici, S. Sailada, and I. B. Gorotxategi, "Evaluating Post-Quantum Cryptography in TLS-Based IoT Telemetry and OPC UA Industrial Communications," in *Proc. 2026 26th International Conference on Transparent Optical Networks*, 2026, doi: 10.1109/ICTON71309.2026.11705248. [Online]. Available: https://doi.org/10.1109/ICTON71309.2026.11705248

[46] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST SP 800-38D, 2007, doi: 10.6028/NIST.SP.800-38D. [Online]. Available: https://doi.org/10.6028/NIST.SP.800-38D

[47] M. S. Muqtada and H. A. M. Elahi, "Archived ESP32 hybrid handshake observations," [dataset], GitHub, 2026, source version 467bbe7a3c5a718d4eec7c882bc00f042487304d. Available: https://github.com/Hybrid-PQC-IoT-Lab/esp32-hybrid-pqc-key-exchange/tree/467bbe7a3c5a718d4eec7c882bc00f042487304d/docs/evidence.

[48] B. Blanchet, "An efficient cryptographic protocol verifier based on Prolog rules," in *Proc. 14th IEEE CSFW*, 2001, pp. 82 to 96, doi: 10.1109/CSFW.2001.930138. [Online]. Available: https://doi.org/10.1109/CSFW.2001.930138